



Transformer

모든 LLM의 기반이 되는 핵심 아키텍처 완전 해부

"Attention Is All You Need" (Vaswani et al., 2017)

목차

- | | | | |
|----|----------------------------|----|-------------------------|
| 01 | Transformer 탄생 배경 | 07 | Encoder 구조 상세 |
| 02 | 전체 아키텍처 개요 | 08 | Decoder 구조 상세 |
| 03 | 입력 표현: 임베딩과 위치 인코딩 | 09 | 학습 전략과 최적화 |
| 04 | Self-Attention 메커니즘 | 10 | Transformer vs RNN/LSTM |
| 05 | Multi-Head Attention | 11 | Transformer 기반 모델 계보 |
| 06 | Feed-Forward Network & 정규화 | 12 | 현대 LLM으로의 진화 |

01 Transformer 탄생 배경

기존 모델의 한계

RNN: 순차 처리 → 병렬화 불가, 학습 속도 느림

LSTM/GRU: 장기 의존성 부분 해결, 여전히 순차적

Seq2Seq: 고정 길이 벡터 병목 (정보 손실)

Attention 추가: 개선되었으나 RNN 구조 의존

Transformer의 혁신

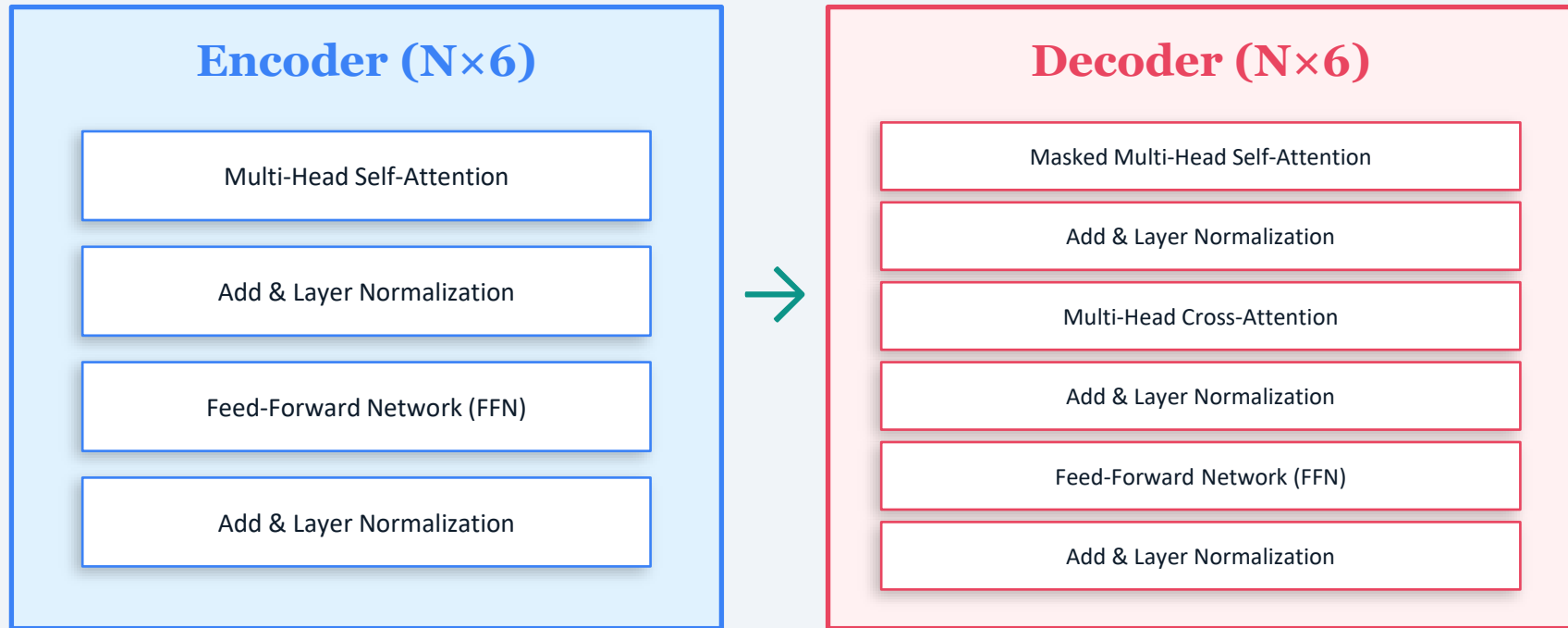
✓ RNN 완전 제거 → 순수 Attention 기반

✓ 완전한 병렬 처리로 학습 속도 혁신

✓ Self-Attention으로 장거리 의존성 직접 포착

✓ 2017 Google Brain 팀, NeurIPS 논문 발표

02 전체 아키텍처 개요



03 입력 표현: 임베딩과 위치 인코딩



Token Embedding

각 토큰을 d_{model} 차원 벡터로 변환
Vocabulary → 학습 가능한 임베딩 행렬
예: "나는" → [0.12, -0.34, 0.56, ...]



Positional Encoding

순서 정보가 없는 Attention에 위치 부여
사인/코사인 함수의 주기적 패턴 활용
학습 가능한 위치 임베딩도 사용 가능

위치 인코딩 수식

$$\text{PE}(\text{pos}, 2i) = \sin(\text{pos} / 10000^{(2i/d_{\text{model}})})$$

$$\text{PE}(\text{pos}, 2i+1) = \cos(\text{pos} / 10000^{(2i/d_{\text{model}})})$$

최종 입력 = Token Embedding + Positional Encoding

04 Self-Attention 메커니즘

Q (Query)

현재 토큰이 다른 토큰에게
'무엇을 찾고 있는지' 질의

K (Key)

각 토큰이 자신을 설명하는
'이름표' 역할

V (Value)

실제 전달할 정보 내용을
담고 있는 벡터

Scaled Dot-Product Attention 계산

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$$

① $Q \cdot K^T$ 유사도 계산

② $\sqrt{d_k}$ 로 스케일링

③ Softmax로 확률 변환

④ V 에 가중합 적용

05 Multi-Head Attention

하나의 Attention이 아닌, 여러 개의 헤드가 각각 다른 관점에서 정보를 추출

Head 1
문법 관계

Head 2
의미 유사도

Head 3
위치 패턴

Head 4
문맥 참조

Head 5
감성 관계

Head 6
구문 구조

Head 7
동의어 매핑

Head 8
지시어 연결

$\text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_8) \rightarrow \text{Linear Projection} \rightarrow \text{MultiHead Output}$

06 Feed-Forward Network & 정규화



Position-wise FFN

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

두 개의 선형 변환 사이에 ReLU 활성화 함수
 $d_{\text{model}}(512) \rightarrow d_{\text{ff}}(2048) \rightarrow d_{\text{model}}(512)$
각 위치(position)에 독립적으로 동일 적용
비선형성을 추가하여 표현력 강화



Layer Norm & Residual

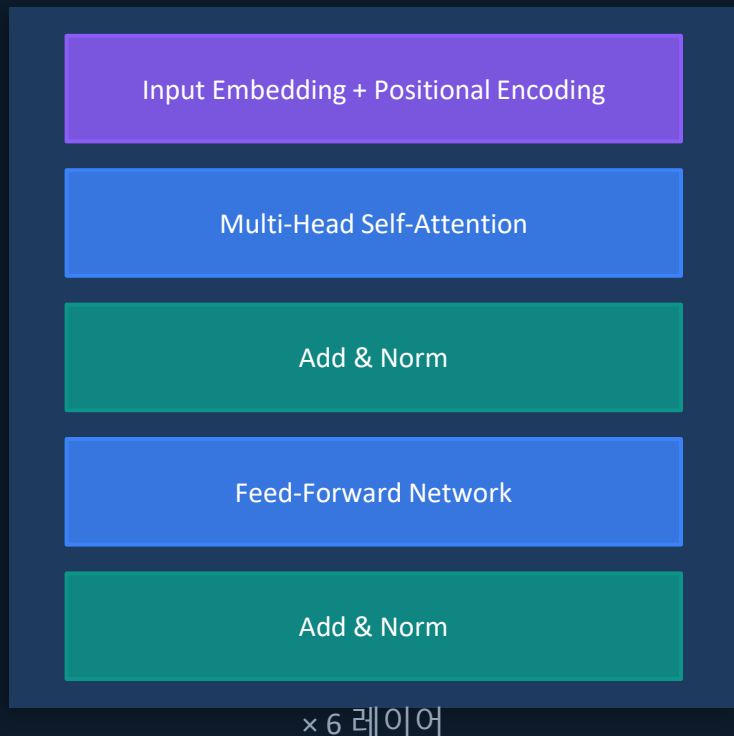
$$\text{Output} = \text{LayerNorm}(x + \text{SubLayer}(x))$$

Residual Connection: 입력을 출력에 직접 더함
→ 기울기 소실 방지, 깊은 네트워크 학습 가능
Layer Normalization: 각 레이어 출력 정규화
→ 학습 안정화, 수렴 속도 향상



Dropout(0.1) 이 각 서브레이어와 임베딩에 적용되어 과적합을 방지합니다.

07 Encoder 구조 상세



핵심 특징

양방향 문맥 이해

모든 토큰이 시퀀스 내 모든 위치를 참조하여 전체 문맥을 동시에 파악합니다.

6개 동일 레이어 스택

각 레이어가 점진적으로 더 높은 수준의 추상 표현을 학습합니다.

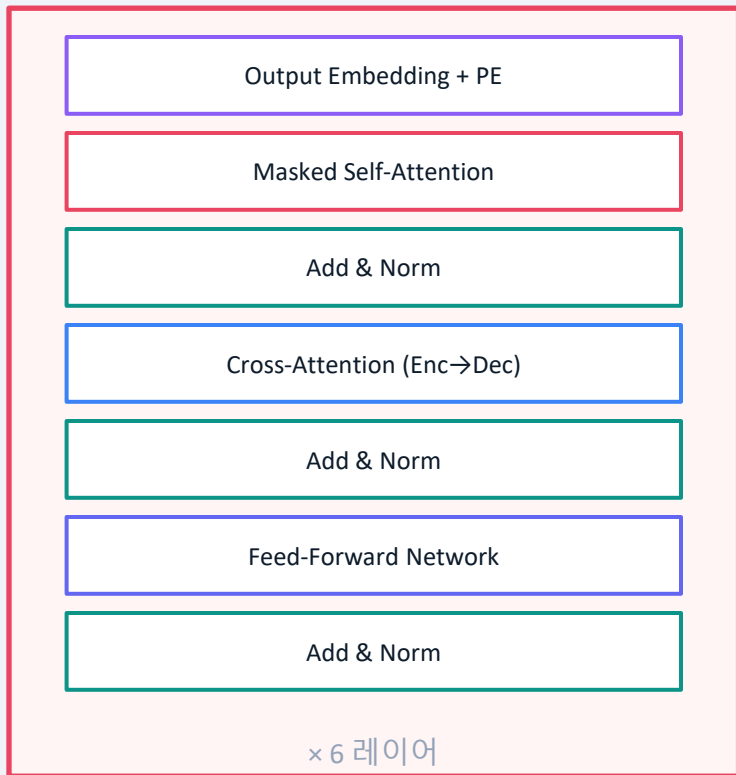
Residual + LayerNorm

스킵 연결로 그래디언트 흐름을 보장하고 정규화로 안정적 학습을 지원합니다.

병렬 처리 가능

Self-Attention은 순서 무관 연산이므로 GPU에서 완전 병렬 실행 가능합니다.

08 Decoder 구조 상세



Masked Self-Attention

미래 토큰을 마스킹하여 자기회귀(auto-regressive) 생성을 보장합니다. Attention 점수에 $-\infty$ 를 설정하여 softmax 후 0으로 만듭니다.

Cross-Attention

Encoder의 출력(K, V)과 Decoder의 현재 상태(Q)를 연결합니다. 디코딩 시 입력 시퀀스의 관련 정보를 참조할 수 있게 합니다.

Auto-Regressive 생성

이전에 생성된 토큰들을 입력으로 받아 다음 토큰을 순차적으로 예측합니다. 추론 시에는 한 번에 하나의 토큰을 생성합니다.

Attention Score 계산 과정

1

선형 변환

입력 x 에 W_Q, W_K, W_V 행렬을 곱하여 Q, K, V 생성

2

유사도 계산

Q 와 K 의 전치행렬을 내적(dot product) 수행

3

스케일링

$\sqrt{d_k}$ 로 나누어 그래디언트 안정화

4

소프트맥스

확률 분포로 변환 (합 = 1)

Attention Matrix 예시 (문장: '나는 학교에 간다')

	나는	학교에	간다
나는	0.65	0.20	0.15
학교에	0.15	0.55	0.30
간다	0.25	0.35	0.40

각 행의 합 = 1.0 (softmax)

높은 값 = 강한 attention (노란색)

대각선이 높음 = 자기 자신 참조 경향

09 학습 전략과 최적화



Teacher Forcing

학습 시 이전 단계의 정답(ground truth)을 디코더 입력으로 사용하여 수렴 속도를 높입니다.



Label Smoothing ($\epsilon=0.1$)

정답 확률을 1.0 대신 0.9로 설정하여 모델의 과신을 방지하고 일반화 성능을 향상시킵니다.



Learning Rate Warmup

학습 초기에 LR을 점진적으로 증가시킨 후 감소합니다.
$$lr = d^{-0.5} \cdot \min(\text{step}^{-0.5}, \text{step} \cdot \text{warmup}^{-1.5})$$



Adam Optimizer

$\beta_1=0.9, \beta_2=0.98, \epsilon=10^{-9}$
적응적 학습률 최적화로 빠른 수렴과 안정적인 학습을 달성합니다.

10 Transformer vs RNN/LSTM

비교 항목	RNN / LSTM	Transformer
처리 방식	순차적(Sequential)	병렬(Parallel)
장거리 의존성	제한적 (기울기 소실)	직접 참조 (Self-Attention)
학습 속도	느림	매우 빠름 (GPU 활용)
시간 복잡도	$O(n)$	$O(n^2)$ (시퀀스 길이)
메모리	효율적	높음 (n^2 attention)
양방향 문맥	BiLSTM 필요	기본 지원 (Encoder)
확장성	제한적	뛰어남 (스케일링 법칙)



핵심: Transformer의 $O(n^2)$ 비용은 스케일링 법칙에 의해 상쇄되며, 큰 모델일수록 성능 향상이 예측 가능합니다.

11 Transformer 기반 모델 계보

Encoder Only

양방향 이해

대표 모델

BERT, RoBERTa,
ALBERT, DeBERTa,
ELECTRA

활용: 분류, NER, QA

Decoder Only

자기회귀 생성

대표 모델

GPT-1/2/3/4,
LLaMA, Claude,
PaLM, Gemini

활용: 텍스트 생성, 대화

Encoder-Decoder

시퀀스 변환

대표 모델

T5, BART,
mT5, FLAN-T5,
Pegasus

활용: 번역, 요약, 변환

2017
Transformer

2018
BERT/GPT-1

2019
GPT-2/T5

2020
GPT-3

2023~
GPT-4/Gemini/Claude

12 현대 LLM으로의 진화

원본 Transformer에서 현대 LLM까지의 핵심 개선 사항

RoPE / ALiBi

절대 위치 인코딩 → 상대적 위치 인코딩으로 전환. 긴 시퀀스 일반화 성능 향상

RMSNorm + Pre-Norm

Layer Norm → RMSNorm, Post-Norm → Pre-Norm으로 학습 안정성 개선

GQA / MQA

Multi-Head → Grouped/Multi-Query Attention으로 추론 효율성 대폭 향상

SwiGLU / GeGLU

ReLU FFN → Gated Linear Units로 교체하여 모델 표현력 향상

Flash Attention

IO-aware 알고리즘으로 메모리 사용량 감소 및 속도 향상

Scaling Laws

Chinchilla 법칙: 모델 크기와 데이터 양의 최적 비율 발견

원본 Transformer 핵심 하이퍼파라미터

파라미터	값	설명
d_model	512	임베딩 및 모델 히든 차원
d_ff	2048	FFN 내부 차원 ($4 \times d_{\text{model}}$)
h (heads)	8	Multi-Head Attention 헤드 수
d_k = d_v	64	각 헤드의 Key/Value 차원 ($512/8$)
N (layers)	6	Encoder/Decoder 각각의 레이어 수
Dropout	0.1	모든 서브레이어에 적용
Warmup Steps	4,000	Learning Rate 워밍업 스텝
총 파라미터	~65M	Base 모델 기준



Summary

Transformer는 단순히 하나의 모델이 아닌, 현대 AI의 기반 아키텍처입니다.

- ✓ Self-Attention으로 순서에 관계없이 전체 문맥을 동시 처리
- ✓ Multi-Head Attention으로 다양한 관점의 정보를 병렬 추출
- ✓ Encoder-Decoder 구조로 이해와 생성을 분리하여 유연성 확보
- ✓ 완전한 병렬화로 대규모 학습이 가능한 확장 가능한 구조